

Examples: Decision Tree Sensitivity, Bootstrapping, and Random Forests

1 Decision Tree Sensitivity to Data Perturbations

1.1 Dataset and Task

We use a small 2-D dataset to classify whether a student **passes** ($y = +1$) or **fails** ($y = -1$) based on two features:

- x_1 : hours studied per day (scale 1–6)
- x_2 : hours slept the previous night (scale 4–8)

Sample	x_1 (study h)	x_2 (sleep h)	y	Label
P_1	1	5	−1	Fail
P_2	2	6	−1	Fail
P_3	3	7	+1	Pass
P_4	4	6	+1	Pass
P_5	5	5	+1	Pass
P_6	6	4	−1	Fail

Table 1: Dataset D_A (original).

Dataset D_B : identical to D_A except that sample P_2 has its label *flipped*: $(x_1, x_2, y) = (2, 6, +1)$.

We grow *fully unpruned* trees (minimise weighted Gini impurity at each split) on both datasets and compare the results.

1.2 Tree A — trained on D_A and Tree B — trained on D_B

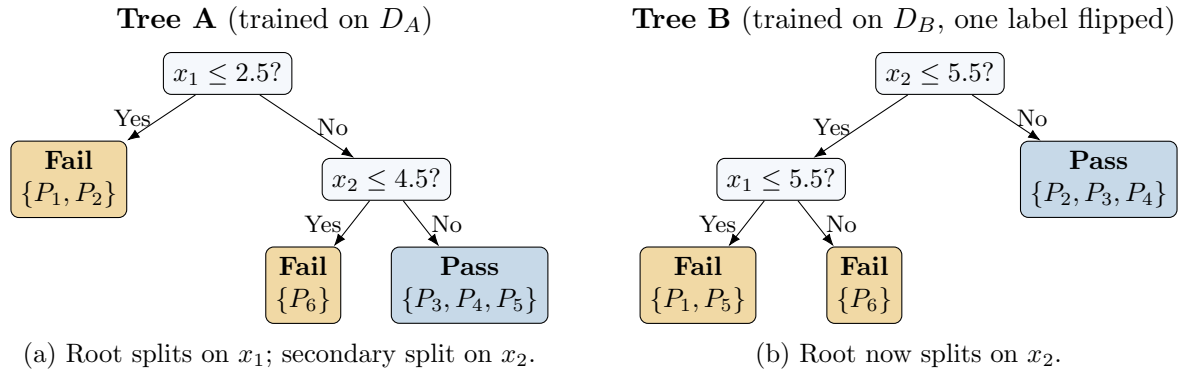


Figure 1: Tree A and Tree B side by side. One label flip (P_2 : Fail $\rightarrow$ Pass) changes the root feature, the threshold, and the entire tree topology.

Key observation: A single label flip changed the root splitting feature (x_1 in Tree A $\rightarrow$ x_2 in Tree B), shifted the threshold, and altered the tree's entire topology. The next section shows what this means for the decision regions.

1.3 Decision Regions: Tree A vs. Tree B

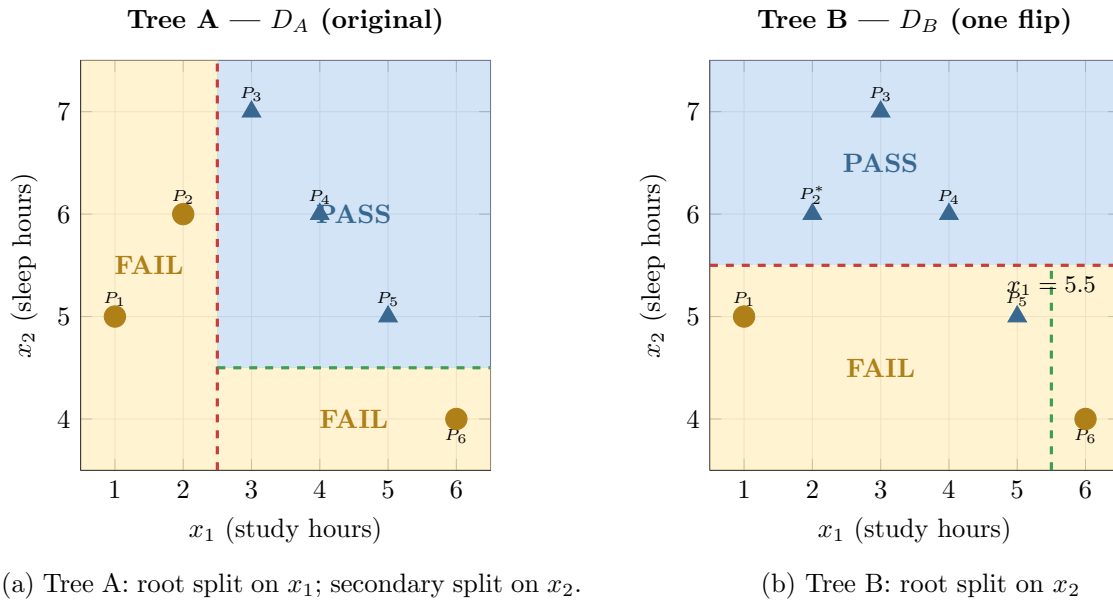


Figure 2: Decision regions from Tree A and Tree B. P_2^* denotes the flipped label. A test point such as (3, 5) is predicted **PASS** by Tree A but **FAIL** by Tree B.

Sensitivity summary. The point (3, 5) (moderate study, moderate sleep) is predicted **Pass** by Tree A (it lies in the blue region: $x_1 > 2.5$, $x_2 > 4.5$) but **Fail** by Tree B (it lies below the $x_2 = 5.5$ boundary). This is *high variance* in action — the prediction for a new point swings based on a single training label. Bagging and Random Forests combat exactly this instability.

Regression Trees

From Classification to Continuous Prediction

1 The BuildTree Algorithm for Regression

The two pseudocodes below are placed side-by-side to make the differences immediately visible. **Orange lines** are the only changes.

Classification BuildTree

```

1: if  $|I| \leq k$  or all labels identical then
2:   return Leaf( $\hat{y} = \text{majority class}$ )
3: end if
4:  $\text{Gini}^* \leftarrow \infty$ 
5: for each  $j$ , threshold  $s$  do
6:    $I^\pm \leftarrow$  partition of  $I$  on  $(j, s)$ 
7:    $c(j, s) \leftarrow \text{wtd. Gini}(I^-, I^+)$ 
8:   if  $c(j, s) < \text{Gini}^*$  then
9:      $(j^*, s^*) \leftarrow (j, s)$ 
10:  end if
11: end for
12: Partition  $I$  on  $(j^*, s^*)$ 
13: return Node( $j^*, s^*, \text{BT}(I^-), \text{BT}(I^+)$ )

```

Regression BuildTree

```

1: if  $|I| \leq k$  or all labels identical then
2:   return Leaf( $\hat{y} = \text{mean}(I)$ )
3: end if
4:  $\text{SSE}^* \leftarrow \infty$ 
5: for each  $j$ , threshold  $s$  do
6:    $I^\pm \leftarrow$  partition of  $I$  on  $(j, s)$ 
7:    $c(j, s) \leftarrow \text{SSE}(I^-) + \text{SSE}(I^+)$ 
8:   if  $c(j, s) < \text{SSE}^*$  then
9:      $(j^*, s^*) \leftarrow (j, s)$ 
10:  end if
11: end for
12: Partition  $I$  on  $(j^*, s^*)$ 
13: return Node( $j^*, s^*, \text{BT}(I^-), \text{BT}(I^+)$ )

```

What Changed — and What Did Not

Changed (2 lines):

- Line 2 — leaf prediction: **majority class** $\rightarrow$ **mean of y values**.
- Line 6 — split criterion: **weighted Gini** $\rightarrow$ **total SSE**.

Unchanged (everything else):

- Stopping condition (line 1): same check on node size / purity.
- Threshold search: same midpoint enumeration over sorted feature values.
- Tie-breaking, recursion, tree structure: all identical.
- Complexity: $O(dm \log m)$ per node, $O(dn \log^2 n)$ for a balanced tree.

2 Full Worked Example: Predicting Student Exam Scores

Student Exam Scores — Running BuildTree for Regression

Setting: 12 students from a previous cohort. We want to *predict* their final exam score ($y \in \mathbb{R}$).

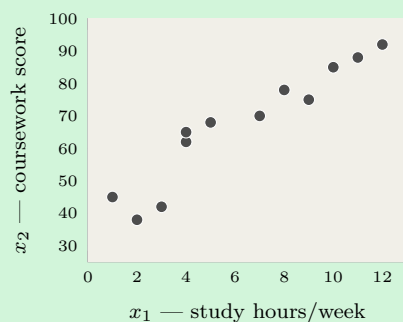
Features: x_1 = study hours per week, x_2 = coursework score (out of 100).

Minimum leaf size: $k = 3$ (stop splitting when a node has ≤ 3 examples).

Student	x_1 (hrs/wk)	x_2 (coursework)	y (exam)
1	2	38	35
2	3	42	40
3	1	45	38
4	4	62	55
5	5	68	60
6	4	65	58
7	7	70	72
8	8	78	78
9	9	75	75
10	11	88	85
11	12	92	90
12	10	85	88

Feature space partition. Figure 1 shows how BUILDTREE carves up the (x_1, x_2) plane step by step.

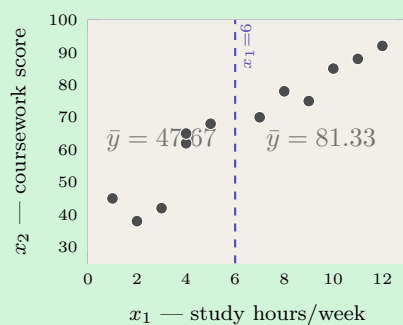
Step 0 — full feature space



Before any split. All 12 students plotted. Overall mean $\bar{y} = 64.5$, $\text{SSE} = 4297.0$.

BUILDTREE tries every feature–threshold combination. The best root split is $x_1 < 6$, reducing SSE from 4297.0 to **896.67** — a reduction of 3400.3.

Step 1 — split $x_1 < 6$



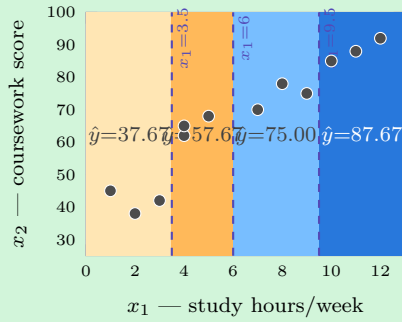
Root split: $x_1 < 6$.

Left: $n = 6$, $\bar{y} = 47.67$, $\text{SSE} = 625.33$

Right: $n = 6$, $\bar{y} = 81.33$, $\text{SSE} = 271.33$

Neither child is pure enough to stop (SSE still large relative to $k = 3$). Both are expanded in Step 2.

Step 2 — final four leaves



Leaf A: $\hat{y} = 37.67$ Leaf B: $\hat{y} = 57.67$ Leaf C: $\hat{y} = 75.00$ Leaf D: $\hat{y} = 87.67$

Two simultaneous splits (left and right branches):

Left ($x_1 < 6$): split $x_1 < 3.5$
SSE: 625.33 $\rightarrow$ 25.33

- $x_1 < 3.5$: $\hat{y} = 37.67$ (Leaf A)
- $x_1 \geq 3.5$: $\hat{y} = 57.67$ (Leaf B)

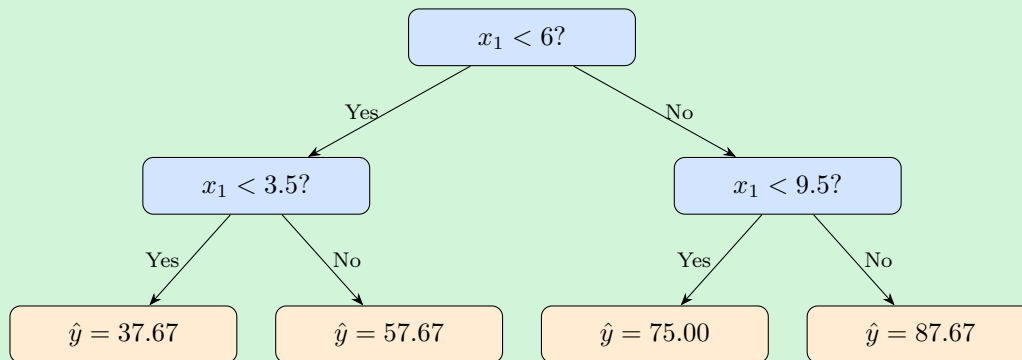
Right ($x_1 \geq 6$): split $x_1 < 9.5$
SSE: 271.33 $\rightarrow$ 30.67

- $x_1 < 9.5$: $\hat{y} = 75.00$ (Leaf C)
- $x_1 \geq 9.5$: $\hat{y} = 87.67$ (Leaf D)

All leaves have $n = 3 = k$ — BUILDTREE stops.

Figure 1: Step-by-step partition of the (x_1, x_2) plane by the regression BUILDTREE. Regions are shaded by leaf prediction $\hat{y}$ (lighter = lower score). Note that x_2 (coursework score) is never selected: x_1 alone reduces SSE sufficiently at every node.

Final regression tree:



Reading off the rules:

1. $x_1 < 3.5$ hrs/week $\rightarrow \hat{y} = 37.67$
2. $3.5 \leq x_1 < 6$ hrs/week $\rightarrow \hat{y} = 57.67$
3. $6 \leq x_1 < 9.5$ hrs/week $\rightarrow \hat{y} = 75.00$
4. $x_1 \geq 9.5$ hrs/week $\rightarrow \hat{y} = 87.67$

Predicting a new student. A student studies 7 hrs/week ($x_1 = 7$), coursework score 73 ($x_2 = 73$):

1. **Root:** Is $x_1 < 6$? $7 < 6$ — **No** $\rightarrow$ go right.
2. **Node:** Is $x_1 < 9.5$? $7 < 9.5$ — **Yes** $\rightarrow$ go left.
3. **Leaf C:** $\hat{y} = 75.00$.

2 Bootstrap Sampling — A Concrete Example

2.1 The Procedure

Given our original dataset $D = \{P_1, P_2, P_3, P_4, P_5, P_6\}$ (6 points), we draw three bootstrap samples, each of size $n = 6$, *with replacement*. Each draw independently selects one of the 6 points with equal probability $\frac{1}{6}$.

2.2 Three Bootstrap Samples

Bootstrap	Draws (with replacement)	Unique pts	OOB pts	Comment
$\tilde{D}^{(1)}$	$P_1, P_3, P_3, P_4, P_5, P_6$	P_1, P_3, P_4, P_5, P_6	P_2	P_3 drawn twice
$\tilde{D}^{(2)}$	$P_2, P_2, P_3, P_4, P_5, P_6$	P_2, P_3, P_4, P_5, P_6	P_1	P_2 drawn twice
$\tilde{D}^{(3)}$	$P_1, P_2, P_3, P_4, P_6, P_6$	P_1, P_2, P_3, P_4, P_6	P_5	P_6 drawn twice

Table 2: Three bootstrap samples. On average, $\approx 63.2\%$ of original points appear in each sample.

2.3 Visual: Which Points Are In / Out?

$\tilde{D}^{(1)}$	P_1 in	P_2 OOB	P_3 in $\times 2$	P_4 in	P_5 in	P_6 in
$\tilde{D}^{(2)}$	OOB	in $\times 2$	in	in	in	in
$\tilde{D}^{(3)}$	in	in	in	in	OOB	in $\times 2$
	in (once)	in (duplicate)	OOB (out-of-bag)			

2.4 OOB Error Estimation

For each point, we can evaluate the trees that did *not* train on it:

- P_2 is OOB for $\tilde{D}^{(1)}$ — evaluate $\text{Tree}^{(1)}$ on P_2 , compare to true label.
- P_1 is OOB for $\tilde{D}^{(2)}$ — evaluate $\text{Tree}^{(2)}$ on P_1 .
- P_5 is OOB for $\tilde{D}^{(3)}$ — evaluate $\text{Tree}^{(3)}$ on P_5 .

Aggregate these OOB predictions to estimate test error — *no held-out validation set required!*

3 Random Forest with Four Trees

3.1 Setup

We train a Random Forest of $B = 4$ trees on a slightly extended dataset (8 points, 2 features) to produce more interesting decision regions.

Sample	x_1	x_2	y
Q_1	1	6	-1 (Fail)
Q_2	2	7	-1 (Fail)
Q_3	3	7	+1 (Pass)
Q_4	4	6	+1 (Pass)
Q_5	5	5	+1 (Pass)
Q_6	6	4	-1 (Fail)
Q_7	4	3	-1 (Fail)
Q_8	3	5	+1 (Pass)

Table 4: Extended dataset for Random Forest example ($d = 2$ features, $m = \lfloor \sqrt{2} \rfloor = 1$ feature sampled per node).

With only $d = 2$ features, we use $m = 1$ (one feature randomly selected at each node).

3.2 Four Bootstrap Samples and Feature Subsampling

Tree	Bootstrap sample	Root: feature drawn / split found
$T^{(1)}$	$Q_1, Q_2, Q_3, Q_4, Q_5, Q_5, Q_7, Q_8$	x_1 drawn $\rightarrow$ best split $x_1 \leq 2.5$
$T^{(2)}$	$Q_1, Q_3, Q_4, Q_5, Q_6, Q_7, Q_8, Q_8$	x_2 drawn $\rightarrow$ best split $x_2 \leq 5.5$
$T^{(3)}$	$Q_2, Q_3, Q_4, Q_5, Q_6, Q_6, Q_7, Q_8$	x_1 drawn $\rightarrow$ best split $x_1 \leq 5.5$
$T^{(4)}$	$Q_1, Q_2, Q_3, Q_4, Q_7, Q_7, Q_8, Q_8$	x_2 drawn $\rightarrow$ best split $x_2 \leq 4.5$

Table 5: Bootstrap samples and root splits for each of the four trees.

3.3 Individual Tree and Forest Decision Regions

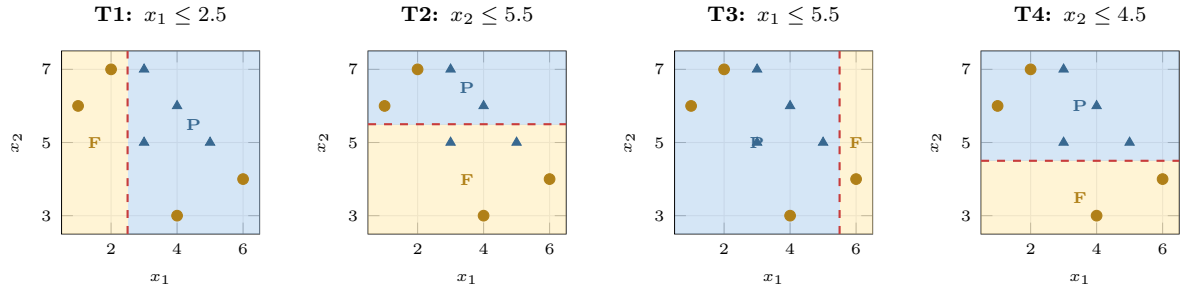
What to observe.

Smoother, step-shaped boundary. The RF boundary is a staircase formed by combining four linear splits. No single tree produces this shape alone.

Reduced variance. The upper-left corner ($x_1 \leq 2.5, x_2 > 5.5$) is predicted **Pass** (3-1 vote) even though Tree 1 alone predicts Fail there. The forest correctly weighs the high sleep hours via the other trees.

Uncertainty zones. The region $2.5 < x_1 \leq 5.5, x_2 \leq 4.5$ produces a 2-2 tie, defaulting to Fail — indicating genuine boundary ambiguity.

Feature diversity matters. Trees 1 & 3 split on x_1 ; Trees 2 & 4 split on x_2 . Without random feature subsampling all trees might converge on the single strongest feature, giving correlated errors and no variance reduction.



Four individual trees ($P = \text{Pass}$, $F = \text{Fail}$; $\bullet = \text{Fail}$, $\blacktriangle = \text{Pass}$). Each tree uses a different split due to bootstrap sampling and random feature subsampling.

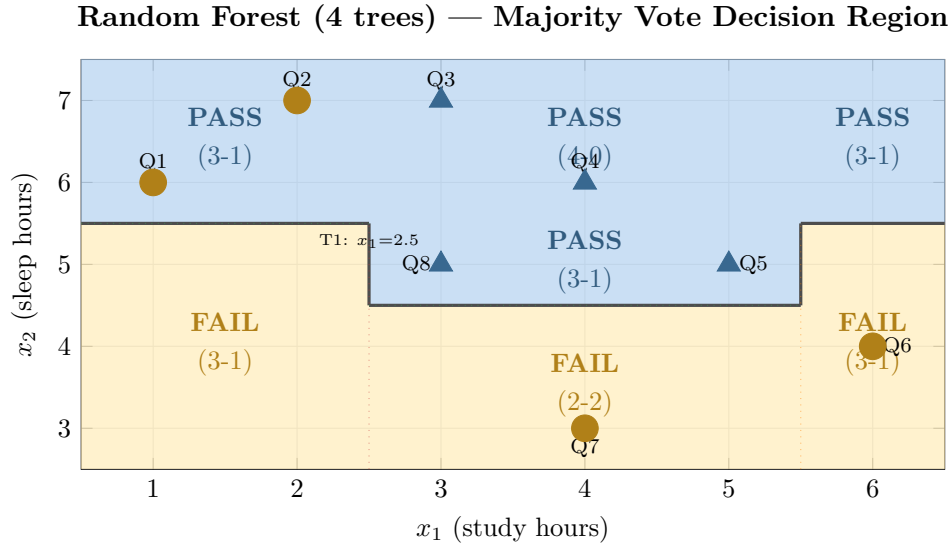


Figure 3: All five decision regions on one page. *Top row:* individual tree regions ($P = \text{Pass}$, $F = \text{Fail}$); each tree uses a single split determined by bootstrap sampling and random feature selection. *Bottom:* Random Forest majority-vote region; vote tallies shown as (Pass–Fail). The staircase boundary (thick black line) is more complex and more robust than any single tree.